PXL Workshop: CI/CD for Hosting Web Applications in AWS

Hands-on session 2: Deploying with Terraform

In this session, you will clone some source code from a GIT repository, start a Terraform deployment, provision infrastructure on AWS and release your application.

0. Prerequisites and setup

- Install Terraform.

```
$ sudo apt-get install terraform

$ terraform --version
Terraform v1.9.6
on linux_amd64
```

- Checkout the source code GIT repository

```
$ cd ~

$ git clone git@github.com:kobethuwis/pxl-workshop-infrastructure-tf.git

$ cd ./pxl-workshop-infrastructure-tf

$ git status
On branch hands-on-session-2
[ … ]
nothing to commit, working tree clean
```

- Give correct permissions to the .sh scripts

```
$ chmod +x ./config_aws.sh
```

- Use the script and provided credentials to configure AWS.

```
$ ./config_aws.sh AWS_ACCESS_KEY AWS_SECRET_ACCESS_KEY
AWS config succeeded
```

1. Initialize your terraform stack

Navigate to the *main.tf* file in the */deployment* directory. This file is used as the 'root' for your terraform stack and contains a set of local variables. Since you will be managing your own infrastructure in this demo, let's write your save your backend in a unique location.

- Update the subdir for the backend config storage location and the full_name parameter in the locals block.

```
terraform {
  backend "s3" {
    bucket = "thuwiko-sandbox-terraform-state"
    key    = "your-name/terraform.tfstate"
    region = "eu-west-1"
  }
}

locals {
  full_name           = "your-name"
  region              = "eu-west-1"
  …
  logs_bucket_id      = "thuwiko-sandbox-logs"
}
```

- Initialize your terraform stack.

```
$ cd ./deployment

$ terraform init
Initializing the backend...
…
Terraform has been successfully initialized!
```

- Do a terraform plan to check if your stack works.

```
$ terraform plan
module.dockerized-web-app.data.aws_ecr_repository.ecr_repository: Reading...
module.dockerized-web-app.data.aws_ami.ami: Reading...
…
module.dockerized-web-app.data.aws_iam_policy_document.iam_policy_document:
Read complete after 0s [id=3910613138]

No changes. Your infrastructure matches the configuration.
```

Your terraform stack is now initialised! During the terraform plan, your codebase requested the state of the resources defined in *modules/dockerized-web-app/data.tf* and added them to the statefile.

2. Provision the security groups & load balancer

Let's get started by provisioning our security groups, access rules and load balancer. This way we can manage the level of access to our application and deploy a secure stack.

- Create 2 *aws_security_group* resources: one for the load balancer and one for your application. Refer to the terraform registry to get you started. Specify a *name_prefix* & use the *vpc_id* variable to provision them in the correct network.
- Do a terraform apply to provision the resources. Write 'yes' if the plan is in line with your provisioned resources. If it fails, troubleshoot your stack and repeat until a successful apply.

```
$ terraform apply
module.dockerized-web-app.data.aws_ami.ami: Reading...
…
Plan: 2 to add, 0 to change, 0 to destroy.
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
…
module.dockerized-web-app.aws_security_group.app_security_group: Creating...
…
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

- Next, specify the required security group rules for the security groups: the load balancer should only be reachable by internal IP's (within the same network) over HTTPS. Outgoing access should be unrestricted. Use a dynamic terraform resource to deploy a rule for each CIDR-block (IP-range within the network).

```
resource "aws_vpc_security_group_ingress_rule"
"lb_security_group_ingress_rule" {
  for_each          = toset(var.cidr_blocks)
  security_group_id = aws_security_group.lb_security_group.id
  from_port         = 443
  to_port           = 443
  ip_protocol       = "tcp"
  description       = "HTTPS traffic"
  cidr_ipv4         = each.value
}

resource "aws_vpc_security_group_egress_rule" "lb_security_group_egress_rule"
{
  security_group_id = aws_security_group.lb_security_group.id
  ip_protocol       = "-1"
  description       = "All outbound traffic"
  cidr_ipv4         = "0.0.0.0/0"
}
```

- Now we need to provision the security group rules for our application security group. Look up the *aws_vpc_security_group_ingress_rule* & *aws_vpc_security_group_egress_rule* in the terraform registry yourself and configure the following:

  - An ingress rule for only the *var.container_port[0]* (the port the container runs on) for each CIDR-block. (use a dynamic resource for this).
  - An egress rule that allows all outbound traffic.

- Let's go ahead and deploy our load balancer. The Application Load Balancer (ALB) will route traffic from the public internet to our private application; we need to deploy it in a public subnet for it to be publicly accessible. The deploy will take a few minutes.

```
resource "aws_lb" "lb" {
  name               = "${var.full_name}-lb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.your-lb-security-group-name.id]
  subnets            = var.lb_subnet_ids

  access_logs {
    bucket  = var.logs_bucket_id
    prefix  = "${var.full_name}-lb/access-logs"
    enabled = true
  }

  connection_logs {
    prefix  = "${var.full_name}-lb/connection-logs"
    bucket  = var.logs_bucket_id
    enabled = true
  }
}
```

- Retrieve the endpoint of your load balancer through your statefile (look for *dns_name*). You can try to open it in a browser, but the access rules will prohibit any external traffic (for now).

```
$ terraform state show module.dockerized-web-app.aws_lb.lb
resource "aws_lb" "lb" {
…
dns_name  = "your-name-web-app-lb-1440135401.eu-west-1.elb.amazonaws.com"
…
}
```

That's it! You've deployed the load balancer successfully and we can go ahead with some more advanced parts of the infrastructure.

3. Provision the task definition & launch template

Specify the machine, resource requirements and access requirements of your application.

- Get started by specifying your task definition. This is the so called blueprint of your container deployment. Specify the docker image, hardware requirements, a basic logging & monitoring setup and the actual ports your container will listen on.

```
resource "aws_ecs_task_definition" "ecs_task_definition" {
  family = var.full_name
  container_definitions = jsonencode([
    {
      name      = "${var.full_name}"
      image     =
"${data.aws_ecr_repository.ecr_repository.repository_url}:${var.docker_image_t
ag}"
      cpu       = 2000
      memory    = 400
      essential = true
      portMappings = [
        {
          containerPort = var.container_ports[0]
          hostPort      = var.container_ports[0]
        }
      ]
      logConfiguration = {
        logDriver = "awslogs"
        options = {
          "awslogs-create-group" = "true"
          "awslogs-group"        = var.full_name
          "awslogs-region"       = var.region
        }
      }
    }
  ])
}
```

- Next up, define the IAM role your machine will use to execute any actions. In the case of your web application, we won't be sending any AWS-related requests or use interfaces hosted in AWS, so we can keep the level of access very basic. You can check out the policy doc in the *data.tf* file, but the configuration of IAM-based access falls outside the scope of this demo.

```
resource "aws_iam_role" "iam_role" {
  name = "${var.full_name}-ecs-iam-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action    = ["sts:AssumeRole"]
        Effect    = "Allow"
        Principal = { "Service" = "ec2.amazonaws.com" }
      },
    ]
  })
}

resource "aws_iam_policy" "iam_policy" {
  name   = "${var.full_name}-ecs-iam-policy"
  policy = data.aws_iam_policy_document.iam_policy_document.json
}

resource "aws_iam_role_policy_attachments_exclusive"
"iam_role_policy_attachment_exclusive" {
  role_name   = aws_iam_role.iam_role.name
  policy_arns = [aws_iam_policy.iam_policy.arn,
"arn:aws:iam::aws:policy/service-role/AmazonEC2ContainerServiceforEC2Role"]

  depends_on = [aws_iam_role.iam_role, aws_iam_policy.iam_policy]
}

resource "aws_iam_instance_profile" "iam_instance_profile" {
  name = "${var.full_name}-ecs-instance-profile"
  role = aws_iam_role.iam_role.name

  depends_on = [aws_iam_role.iam_role]
}
```

- Specify a launch template which acts as the boot instructions of your machine.

```
resource "aws_launch_template" "launch_template" {
  name                   = "${var.full_name}-launch-template"
  image_id               = data.aws_ami.ami.id
  key_name               = var.ec2_key_name
  update_default_version = true
  vpc_security_group_ids = [aws_security_group.app_security_group.id]
  user_data              =
base64encode(templatefile("${path.module}/user_data.sh", { cluster_name =
var.full_name }))

  iam_instance_profile {
    name = aws_iam_instance_profile.iam_instance_profile.name
  }

  monitoring {
    enabled = true
  }

  tag_specifications {
    resource_type = "instance"
    tags = merge(
      { "Name" = var.full_name }
    )
  }
}
```

- Finally, create an actual AWS ECS cluster. The cluster will acts as a wrapper for all the infrastructure hosted for the deployment of your application: use the terraform registry to help you out.

Apply all of the resources you've written and we're good to go.

4. Provision the actual infrastructure

Go ahead and deploy the machines, autoscaling group and the bridges between the load balancer and your containers.

- First, provision an autoscaling group that spins up a single instance, with a maximum of 2 (for blue/green deployment). Use your launch template, and the application security group. Notice that we use t3.nano & t3a.nano instances, which each have 2vCPU and 512Mb RAM (in line with the container task definition resource request).

```
resource "aws_autoscaling_group" "autoscaling_group" {
  desired_capacity      = 0
  max_size              = 2
  min_size              = 1
  vpc_zone_identifier   = var.app_subnet_ids
  protect_from_scale_in = true

  mixed_instances_policy {
    instances_distribution {
      spot_allocation_strategy = "capacity-optimized"
    }

    launch_template {
      launch_template_specification {
        launch_template_id = aws_launch_template.launch_template.id
        version            =
aws_launch_template.launch_template.latest_version
      }
      override {
        instance_type = "t3a.nano"
      }
      override {
        instance_type = "t3.nano"
      }
    }
  }

  lifecycle {
    ignore_changes = [
      desired_capacity,
      tag
    ]
  }

depends_on = [aws_ecs_cluster.your_ecs_cluster_name]
}
```

AWS EC2 will now send a spot request for 1 instance on our behalf, and the actual machine will be provisioned. Let's link our load balancer to the effective machines.

- Define a target group for our load balancer, which is an instruction on how to access your machine. This will only make the instances accessible via the specific port.

```
resource "aws_lb_target_group" "lb_target_group" {
  name     = "${var.full_name}-tg"
  port     = var.container_ports[0]
  protocol = "HTTP"
  vpc_id   = var.vpc_id

  health_check {
    port     = var.container_ports[0]
    matcher  = "200,404"
    protocol = "HTTP"
  }
}
```

- Link your target group to the autoscaling group by using the *aws_autoscaling_attachment* resource. Let the terraform registry help you out.
- Extend the stack by listing the autoscaling group as the capacity provider for your ECS cluster.

```
resource "aws_ecs_capacity_provider" "ecs_capacity_provider" {
  name = "${var.full_name}-ecs-capacity-provider"

  auto_scaling_group_provider {
    auto_scaling_group_arn         =
aws_autoscaling_group.autoscaling_group.arn
    managed_termination_protection = "ENABLED"

    managed_scaling {
      maximum_scaling_step_size = 2
      minimum_scaling_step_size = 1
      status                    = "ENABLED"
      target_capacity           = 100
    }
  }
}

resource "aws_ecs_cluster_capacity_providers" "ecs_cluster_capacity_providers"
{
  cluster_name = aws_ecs_cluster.ecs_cluster.name

  capacity_providers = [aws_ecs_capacity_provider.ecs_capacity_provider.name]

  default_capacity_provider_strategy {
    base              = 1
    weight            = 100
    capacity_provider = aws_ecs_capacity_provider.ecs_capacity_provider.name
  }
}
```

ECS will manage the scaling of the machines for you (within the parameters specified earlier) based on the number of requested tasks in the service.

The machines are up, the task definition are written and our load balancer is live. All there's left is to request the actual deployment of our container behind the load balancer.

- Create a listener on the load balancer that forwards traffic accordingly. We want to forward HTTPS traffic from port 443 to the target group of the provisioned machines.
- Finalize the setup by requesting a deployment of an ECS service. Let's create 2 copies of your container to effectively load balance incoming requests.

```
resource "aws_ecs_service" "ecs_service" {
  name            = "${var.full_name}-ecs-service"
  cluster         = aws_ecs_cluster.ecs_cluster.id
  task_definition = aws_ecs_task_definition.ecs_task_definition.id
  desired_count   = 2

  load_balancer {
    target_group_arn = aws_lb_target_group.lb_target_group.arn
    container_name   = var.full_name
    container_port   = var.container_ports[0]
  }

  lifecycle {
    ignore_changes = [
      capacity_provider_strategy,
      task_definition
    ]
  }
}
```

The service is now created and the cluster will upscale your infrastructure to 2 running machines and thus send out an additional spot request.

Congratulations, you've just hosted your web-app in a robust, scalable and dynamic deployment.

Notify one of the speakers that you've completed the hands-on session. We'll open the security groups to your load balancers so anyone in the workshop can use your web-app!